GLUTEN FREE LIFE

DOCUMENTACIÓN TÉCNICA

Manual técnico

Arquitectura, módulos, base de datos, ejecución y pruebas del sistema de gestión y punto de venta.

Sistema de gestión y punto de venta
para comercios de productos sin TACC
Para desarrolladores y mantenimiento · Noviembre de 2026
Override Software Solutions · «Del barrio, para el barrio»

Contenido

1 Introducción general

2 Arquitectura del sistema

3 Tecnologías y versiones

4 Estructura del proyecto

5 Base de datos

6 Configuración e instalación

7 Componentes principales

8 Flujo técnico de una venta

9 Gestión de errores y validaciones

10 Pruebas implementadas

11 Datos de demostración

12 Consideraciones de mantenimiento

1. Introducción general

Gluten Free Life es un sistema de gestión y punto de venta para comercios de productos sin TACC, desarrollado por el equipo **Override Software Solutions** («Del barrio, para el barrio») para el cliente *Gluten Free Life*. Reemplaza la caja que solo cobra, las planillas y los cuadernos por una aplicación web única.

Objetivo. Este documento describe la arquitectura, los módulos principales, la lógica de negocio, la base de datos, la instalación, la ejecución, el manejo de errores y las pruebas, de modo que cualquier persona con conocimientos de desarrollo pueda ejecutar el sistema y continuar su evolución sin depender de los desarrolladores originales.

Alcance. El sistema cubre:

- Punto de venta con lector de código de barras, productos por unidad y a granel, medios de pago y tickets.
- Inventario, precios (costo, margen, precio sugerido) y baja lógica de productos.
- Vencimientos por producto, con descuento por el vencimiento más próximo.
- Compras: pedido único al intermediario logístico con 7 estados, exportación a PDF y Excel, envío por WhatsApp o correo, e ingreso de mercadería.
- Caja (arqueo y cierre diario), reportes e importación de catálogos por Excel.
- Operación sin conexión de las ventas (PWA + IndexedDB) y sincronización automática.

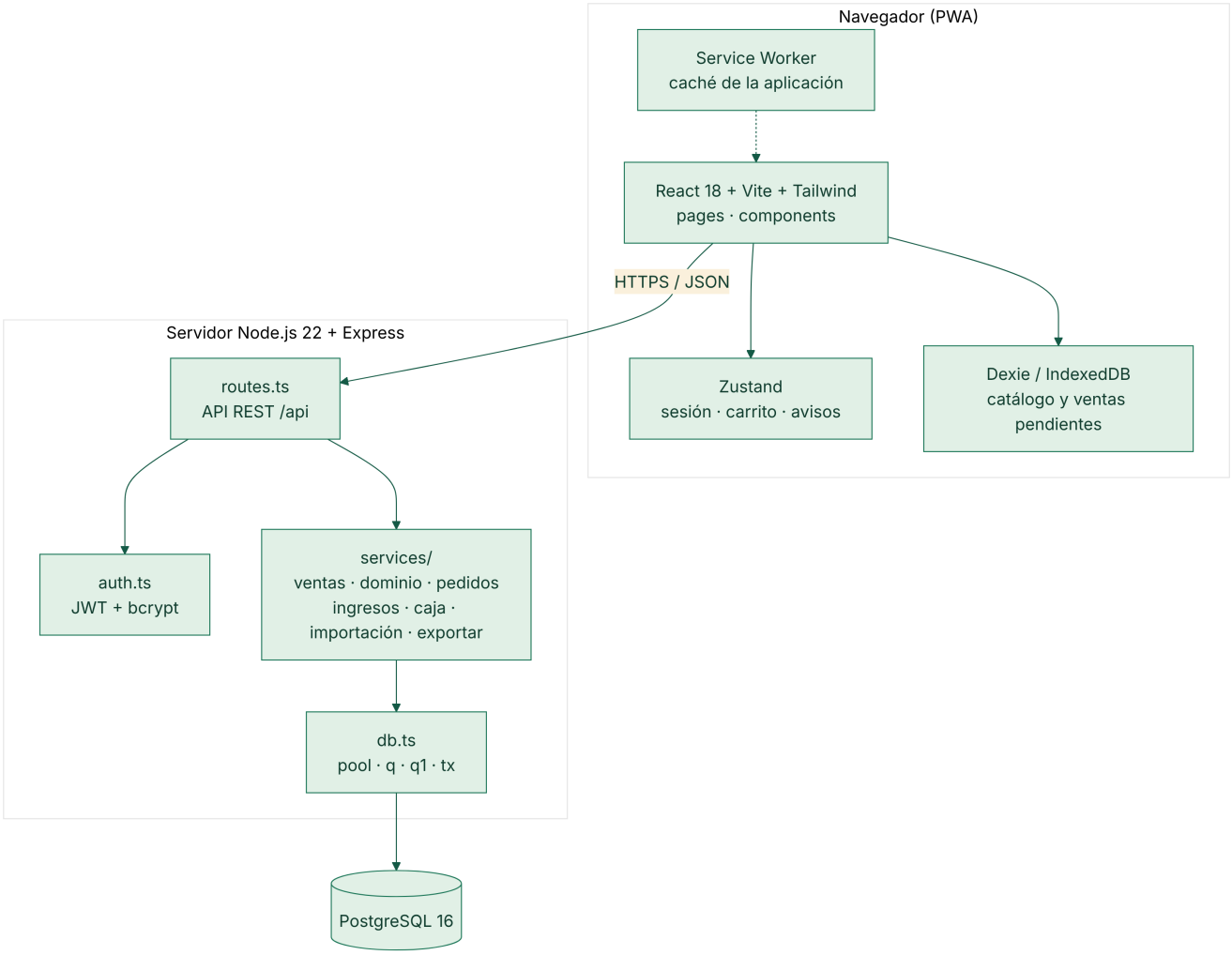
Fuera de alcance (límites conocidos). Un solo usuario sin roles; Mercado Pago y Tarjeta se registran como medio de pago pero no hay integración con la pasarela; no existe réplica remota automática de la base (solo copia de seguridad manual); la sincronización offline es solo de ventas.

Decisiones de modelado: sin lotes (vencimiento por producto); descuento por el vencimiento más próximo; un único intermediario logístico; pedido con 7 estados lineales; entidades `ingreso_mercaderia` y `detalle_ingreso`.

2. Arquitectura del sistema

El sistema utiliza una arquitectura **full-stack en capas**, cliente–servidor, con una API REST entre el navegador y el servidor:

Capa	Ubicación	Responsabilidad
Presentación	`frontend/src/pages`, `components`	Pantallas, formularios, navegación
Estado y datos locales	`frontend/src/store`, `lib/localdb.ts`, `lib/sync.ts`	Sesión, carrito, caché offline, cola de ventas pendientes
Cliente HTTP	`frontend/src/lib/api.ts`	Llamadas a la API, manejo de errores y de red
API REST	`backend/src/routes.ts`, `auth.ts`, `http.ts`	Endpoints, autenticación JWT, validación, errores
Dominio / servicios	`backend/src/services/*`	Reglas de negocio R1–R12, dentro de transacciones
Acceso a datos	`backend/src/db.ts`	Pool de conexiones, consultas y transacciones
Persistencia	PostgreSQL 16	Datos del negocio



Arquitectura: el navegador (PWA) consume la API REST del servidor Node.js, que accede a PostgreSQL.

Esquema de componentes en texto:

```
Navegador
   │
   ├── Frontend React + TypeScript + Vite (PWA)
   │       ├── Interfaz y navegación
   │       ├── Estado global
   │       ├── IndexedDB para operación offline
   │       └── Cliente HTTP de la API
   │
   └── Backend Node.js + TypeScript + Express
           ├── Autenticación (JWT) — cuenta única, sin roles
           ├── Rutas REST
           ├── Servicios de dominio
           ├── Validación de entrada
           └── PostgreSQL
```

En Docker, el backend sirve también el frontend compilado (`frontend/dist`). El archivo `docker-compose.yml` levanta dos servicios:

- `db` : PostgreSQL 16 (`postgres:16-alpine`).
- `app` : imagen que compila y ejecuta frontend y backend.

3. Tecnologías y versiones

Frontend

Tecnología	Versión	Uso
React	18.3	Interfaz
TypeScript	5.5	Lenguaje
Vite	5.4	Compilación y servidor de desarrollo
React Router	6.26	Rutas
Zustand	4.5	Estado global
Tailwind CSS	3.4	Estilos
Dexie (IndexedDB)	4.0	Caché y ventas offline
vite-plugin-pwa	0.20	Service worker (PWA)
xlsx (SheetJS)	0.18	Lectura de planillas
Recharts	2.12	Gráficos
lucide-react	0.447	Íconos

Backend

Tecnología	Versión	Uso
Node.js	22	Entorno de ejecución
TypeScript	5.5	Lenguaje
Express	4.19	Servidor HTTP
PostgreSQL	16	Base de datos
`pg`	8.12	Acceso a datos (SQL explícito, sin ORM)
Zod	3.23	Validación
bcryptjs	2.4	Hash de contraseñas
jsonwebtoken	9.0	Sesiones (JWT)
ExcelJS / PDFKit	4.4 / 0.15	Exportación a Excel y PDF
tsx	4.19	Ejecución de TypeScript en desarrollo y pruebas

4. Estructura del proyecto

```
frontend/
├── src/App.tsx                Rutas y protección de pantallas
├── src/components/            Componentes reutilizables (ui, Layout, Ticket, ProductoForm)
├── src/pages/                 Pantallas funcionales
├── src/lib/api.ts             Cliente de la API
├── src/lib/localdb.ts         Persistencia local
├── src/lib/sync.ts            Monitor y sincronización offline
└── src/store/stores.ts        Estado global

backend/
├── src/index.ts               Configuración e inicio de Express
├── src/routes.ts              Endpoints REST
├── src/auth.ts                Autenticación y JWT
├── src/db.ts                  Pool, consultas y transacciones
├── src/http.ts                Errores HTTP
├── src/migrate.ts             Ejecución de migraciones
├── src/seed.ts                Datos de demostración
├── src/services/dominio.ts    Reglas de productos y vencimientos
├── src/services/ventas.ts     Registro y anulación de ventas
├── src/services/caja.ts       Resumen, cierre y reportes de caja
├── src/services/pedidos.ts    Pedidos de compra
├── src/services/ingresos.ts   Ingreso de mercadería
├── src/services/importacion.ts Importación y validación de catálogos
├── src/services/exportar.ts   Exportación de archivos (PDF y Excel)
└── src/test/e2e.ts            Prueba de integración de la API

backend/migrations/
└── 001_init.sql               Esquema inicial de PostgreSQL

Dockerfile · docker-compose.yml · .env.example
```

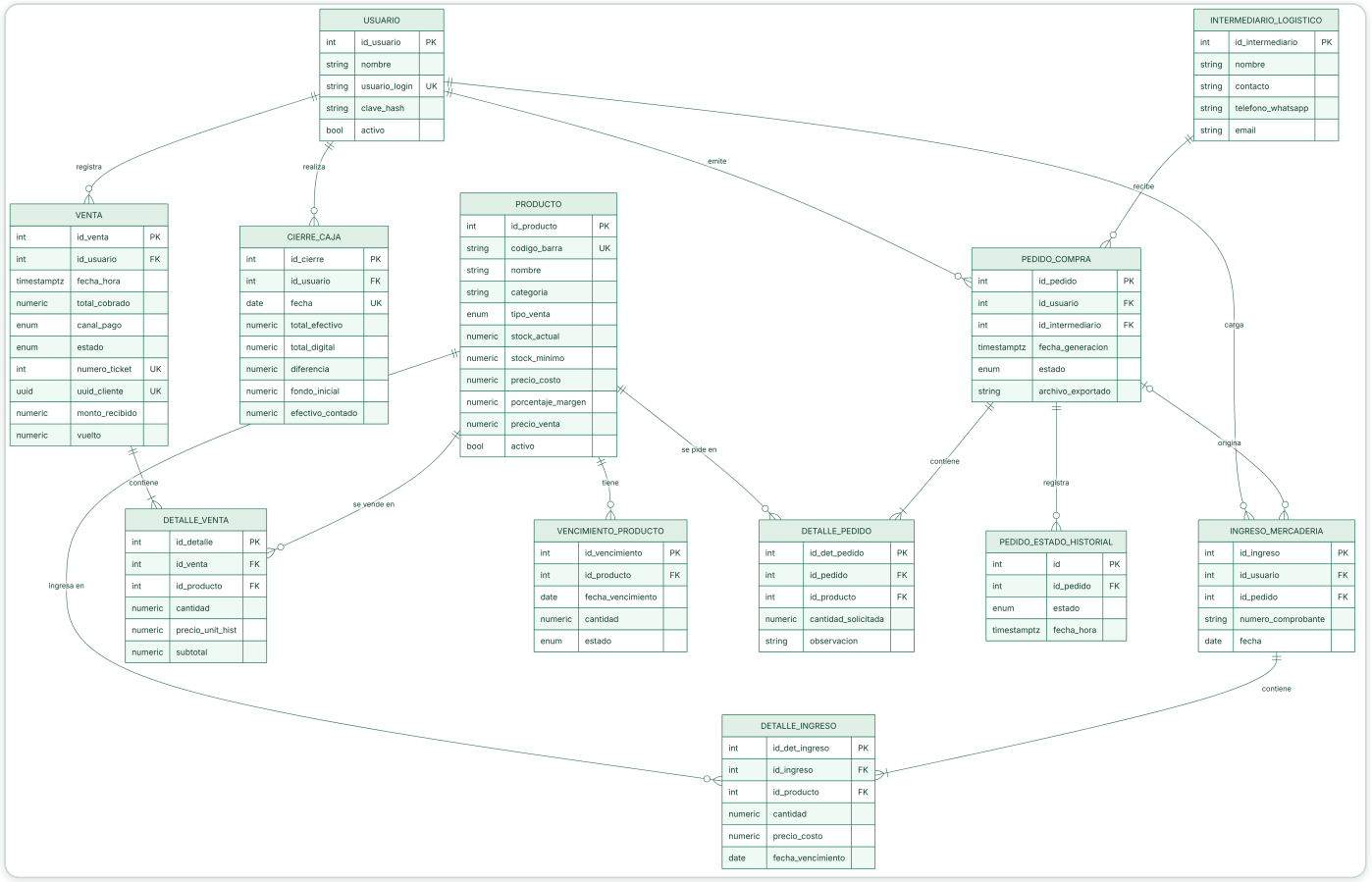
El sistema es una aplicación web (no usa Activities de Android): las «vistas» son las páginas de React (`frontend/src/pages`) y los componentes reutilizables.

Página	Función
`Login.tsx`	Ingreso al sistema
`Dashboard.tsx`	Tablero inicial
`POS.tsx`	Carrito y registro de ventas
`Inventario.tsx`, `ProductoDetalle.tsx`, `Productos.tsx`	Administración de productos
`Vencimientos.tsx`, `StockCritico.tsx`	Seguimiento de vencimientos y reposición
`Compras.tsx`, `PedidoDetalle.tsx`, `IngresoMercaderia.tsx`, `Intermediario.tsx`	Compras e ingresos
`Caja.tsx`, `Reportes.tsx`	Cierre diario e indicadores
`Importar.tsx`, `Configuracion.tsx`	Importación de catálogo y configuración

5. Base de datos

La migración se encuentra en `backend/migrations/001_init.sql`. El motor es PostgreSQL 16.

5.1 Diagrama Entidad–Relación (DER)



DER generado a partir del esquema real. PK: clave primaria, FK: clave foránea, UK: valor único. Los campos «técnicos» agregados al modelo original se incluyen en la descripción de tablas.

5.2 Descripción de tablas y campos

Tipos enumerados: `tipo_venta` (UNIDAD, GRANEL), `canal_pago` (EFECTIVO, MERCADO_PAGO, TARJETA), `estado_venta` (REGISTRADA, ANULADA), `estado_vencimiento` (NORMAL, PROXIMO, VENCIDO, RETIRADO), `estado_pedido` (EN_PREPARACION, GENERADO, EXPORTADO, ENVIADO, EN_GESTION, ENTREGADO, FINALIZADO).

`usuario` — cuenta de acceso (única; registra quién hizo cada operación).

Campo	Tipo	Descripción
`id_usuario`	SERIAL, PK	Identificador
`nombre`	VARCHAR(120)	Nombre visible
`usuario_login`	VARCHAR(60), único	Usuario para ingresar
`clave_hash`	VARCHAR(200)	Contraseña con hash bcrypt
`activo`	BOOLEAN	Habilita o deshabilita la cuenta

`producto` — catálogo.

Campo	Tipo	Descripción
`id_producto`	SERIAL, PK	Identificador
`codigo_barra`	VARCHAR(60), único	Código para el lector
`nombre`, `categoria`	VARCHAR	Datos del producto
`tipo_venta`	`tipo_venta`	UNIDAD o GRANEL
`stock_actual`	NUMERIC(12,3), CHECK ≥ 0	Stock (kg si es granel)
`stock_minimo`	NUMERIC(12,3)	Umbral de stock crítico
`precio_costo`	NUMERIC(12,2)	Último costo
`porcentaje_margen`	NUMERIC(5,2)	Margen comercial (%)
`precio_venta`	NUMERIC(12,2)	Precio al público
`activo`	BOOLEAN	Baja lógica

`vencimiento_producto` — vencimiento por producto (sin lotes).

Campo	Tipo	Descripción
`id_vencimiento`	SERIAL, PK	Identificador
`id_producto`	INTEGER, FK → producto (CASCADE)	Producto
`fecha_vencimiento`	DATE	Fecha
`cantidad`	NUMERIC(12,3), CHECK ≥ 0	Cantidad que vence ese día
`estado`	`estado_vencimiento`	NORMAL, PROXIMO, VENCIDO o RETIRADO

`venta` y `detalle_venta`

Campo	Tipo	Descripción
`venta.id_venta`	SERIAL, PK	Identificador
`venta.id_usuario`	FK → usuario	Quién vendió
`venta.fecha_hora`	TIMESTAMPTZ	Momento de la venta
`venta.total_cobrado`	NUMERIC(12,2)	Total
`venta.canal_pago`	`canal_pago`	Medio de pago
`venta.estado`	`estado_venta`	REGISTRADA o ANULADA
`venta.numero_ticket`	SERIAL, único	Número de ticket
`venta.uuid_cliente`	UUID, único	Identificador generado en el navegador (evita duplicados al sincronizar)
`venta.monto_recibido`, `vuelto`	NUMERIC(12,2)	Pago en efectivo
`venta.sincronizada_en`	TIMESTAMPTZ	Si vino de la cola offline
`detalle_venta.id_detalle`	SERIAL, PK	Identificador
`detalle_venta.id_venta`	FK → venta (CASCADE)	Venta
`detalle_venta.id_producto`	FK → producto	Producto vendido
`detalle_venta.cantidad`	NUMERIC(12,3), CHECK > 0	Cantidad
`detalle_venta.precio_unit_hist`	NUMERIC(12,2)	Precio al momento de la venta
`detalle_venta.subtotal`	NUMERIC(12,2)	Cantidad × precio

`cierre_caja` — un cierre por día.

Campo	Tipo	Descripción
`id_cierre`	SERIAL, PK	Identificador
`id_usuario`	FK → usuario	Quién cerró
`fecha`	DATE, único	Día cerrado
`total_efectivo`, `total_digital`	NUMERIC(12,2)	Totales por medio de pago
`diferencia`	NUMERIC(12,2)	Contado − esperado
`fondo_inicial`, `efectivo_contado`	NUMERIC(12,2)	Datos del arqueo
`observacion`, `creado_en`	TEXT, TIMESTAMPTZ	Nota y fecha de registro

`intermediario_logistico`: `id_intermediario` (PK), `nombre`, `contacto`, `telefono_whatsapp`, `email`, `observaciones`.

`pedido_compra`

Campo	Tipo	Descripción
`id_pedido`	SERIAL, PK	Identificador
`id_usuario`	FK → usuario	Quién lo generó
`id_intermediario`	FK → intermediario_logistico	Destinatario
`fecha_generacion`, `actualizado_en`	TIMESTAMPTZ	Fechas
`estado`	`estado_pedido`	Uno de los 7 estados
`archivo_exportado`	VARCHAR(200)	Constancia del archivo generado

`detalle_pedido`: `id_det_pedido` (PK), `id_pedido` (FK, CASCADE), `id_producto` (FK), `cantidad_solicitada` (> 0), `observacion`; único `(id_pedido, id_producto)`.

`pedido_estado_historial`: `id` (PK), `id_pedido` (FK, CASCADE), `estado`, `fecha_hora`.

`ingreso_mercaderia`: `id_ingreso` (PK), `id_usuario` (FK), `id_pedido` (FK opcional), `numero_comprobante`, `fecha`.

`detalle_ingreso`: `id_det_ingreso` (PK), `id_ingreso` (FK, CASCADE), `id_producto` (FK), `cantidad` (> 0), `precio_costo`, `fecha_vencimiento` (opcional).

`configuracion`: `clave` (PK), `valor`. Claves: `comercio_nombre`, `comercio_direccion`, `comercio_telefono`, `ticket_texto`, `stock_minimo_default`, `umbral_alerta`, `margen_default`, `dias_alerta_vencimiento`, `impresion_ancho_ticket`.

`alerta_sistema`: `id` (PK), `tipo`, `mensaje`, `fecha_hora`, `leida`. Registra avisos como el stock insuficiente al sincronizar ventas offline.

5.3 Relaciones

```
usuario 1 ──── N venta
usuario 1 ──── N cierre_caja
usuario 1 ──── N pedido_compra
usuario 1 ──── N ingreso_mercaderia

producto 1 ──── N vencimiento_producto
producto 1 ──── N detalle_venta
producto 1 ──── N detalle_pedido
producto 1 ──── N detalle_ingreso

venta 1 ──── N detalle_venta
pedido_compra 1 ──── N detalle_pedido
pedido_compra 1 ──── N pedido_estado_historial
pedido_compra 1 ──── N ingreso_mercaderia   (el pedido es opcional en el ingreso)
ingreso_mercaderia 1 ──── N detalle_ingreso
intermediario_logistico 1 ──── N pedido_compra
```

Las relaciones se implementan con claves foráneas. Los detalles de ventas, pedidos, ingresos e historial utilizan eliminación en cascada cuando corresponde.

5.4 Integridad

- Claves primarias autoincrementales.
- Restricciones `NOT NULL`.
- Valores únicos para login, código de barras, número de ticket, `uuid_cliente` y fecha de cierre de caja.

- `CHECK` para evitar stocks y cantidades negativas.
- Enumeraciones para tipos de venta, medios de pago y estados.
- Índices para búsquedas por producto, fecha y relación.

6. Configuración e instalación

6.1 Tecnologías y versiones requeridas

Componente	Versión	Necesario para
Docker Engine + Docker Compose	actuales (Compose V2 o `docker-compose`)	Instalación con Docker
Node.js	22	Desarrollo local
npm	10 o superior	Desarrollo local
PostgreSQL	16	Desarrollo local
Git	cualquiera	Obtener el proyecto
Navegador	Chrome, Edge o Firefox actuales	Usar el sistema

6.2 Dependencias

Se instalan con `npm install` (o `npm ci`) en `backend/` y `frontend/` a partir de sus `package.json`; las versiones están detalladas en la sección 3. Con Docker se instalan automáticamente al construir la imagen.

6.3 Variables de entorno

En Docker se configura mediante el archivo `.env` (copiar de `.env.example`):

Variable	Uso	Valor habitual
`POSTGRES_PASSWORD`	Contraseña de PostgreSQL	`gfl_dev`
`JWT_SECRET`	Firma de tokens	valor de desarrollo
`TZ_NEGOCIO`	Zona horaria comercial	`America/Argentina/Buenos_Aires`
`PORT`	Puerto del backend	`4000`
`APP_PORT`	Puerto publicado por Docker	`4000`
`CORS_ORIGIN`	Origen permitido	`http://localhost:4000` con Docker
`STORAGE_DIR`	Archivos generados	`/data/storage` en Docker
`DATABASE_URL`	Conexión a PostgreSQL (desarrollo local)	`postgres://usuario:clave@localhost:5432/gfl`

Para desarrollo local, el backend utiliza `backend/.env` y el frontend puede apuntar a otro backend mediante `VITE_API_TARGET`. **En producción** definir un `JWT_SECRET` largo y propio (por ejemplo `openssl rand -hex 32`); el backend advierte por consola si no está definido.

6.4 Ejecución con Docker

Requisitos:

- Docker Engine.
- Docker Compose.
- Git, si el proyecto se obtiene clonando el repositorio.

En Linux, el usuario debe tener permisos para acceder al socket de Docker. La lista de requisitos del entorno también está disponible en `requirements.txt`. Este archivo es únicamente documentación: el proyecto no utiliza Python ni se instala con `pip`.

Pasos desde la raíz del proyecto:

```
cp .env.example .env        # cambiar POSTGRES_PASSWORD y JWT_SECRET
docker-compose up -d --build
```

Con Docker Compose moderno también puede utilizarse:

```
docker compose up -d --build
```

Si Docker devuelve `PermissionError: [Errno 13] Permission denied`, el usuario actual no tiene permisos sobre el servicio Docker. En ese caso, ejecutar:

```
sudo docker-compose up -d --build
```

Con Compose V2:

```
sudo docker compose up -d --build
```

Para configurar el acceso permanente sin `sudo`:

```
sudo usermod -aG docker "$USER"
sudo reboot
```

Después del reinicio se puede comprobar el acceso con:

```
docker ps
```

La aplicación se abre en `http://localhost:4000`. Al iniciar, el contenedor aplica las migraciones, carga los datos de demostración **solo si la base está vacía** y arranca el servidor.

6.5 Desarrollo local (importar y ejecutar el proyecto)

1. Obtener el proyecto (`git clone` o descomprimir el `.zip`) y abrir la carpeta en el editor (por ejemplo VS Code).
2. Crear la base de datos (ver 6.6).
3. Backend:

```
cd backend
cp .env.example .env        # ajustar DATABASE_URL
npm install
npm run migrate
npm run seed
npm run dev
```

1. Frontend, en otra terminal:

```
cd frontend
npm install
npm run dev
```

El frontend se abre en `http://localhost:5173` y utiliza el proxy `/api` hacia `http://localhost:4000`.

Para producción sin Docker: `npm run build` en ambas carpetas y `npm start` en `backend/`; el backend sirve el frontend compilado si existe `frontend/dist`.

Scripts del backend: `dev`, `build`, `start`, `migrate`, `seed`, `migrate:prod`, `seed:prod`, `typecheck`, `test`.

6.6 Crear o restaurar la base de datos

Crear (instalación nueva):

```
createdb gfl
cd backend && npm run migrate     # crea tablas, tipos e índices (001_init.sql)
npm run seed                      # opcional: datos de demostración (BORRA los datos de negocio)
```

`npm run migrate` es idempotente: aplica solo las migraciones pendientes y avisa «Base de datos al día» si no hay ninguna.

Respaldar y restaurar con PostgreSQL:

```
# Respaldo
pg_dump -Fc -U postgres gfl > gfl.dump
# Con Docker
docker compose exec db pg_dump -Fc -U postgres gfl > gfl.dump

# Restauración sobre una base vacía
createdb gfl_restaurada
pg_restore -U postgres -d gfl_restaurada gfl.dump
```

El botón **Configuración → Copia de seguridad** y el endpoint `GET /api/sistema/backup` descargan un archivo JSON con los datos del negocio útil para conservar o inspeccionar información; **no reemplaza** a `pg_dump` y no tiene un procedimiento de restauración automático.

6.7 Credenciales de prueba

Usuario	Contraseña	Alcance
`admin`	`1234`	Cuenta única de demostración con acceso total

Solo para demostración: **cambiar la contraseña** en *Configuración → Cuenta de acceso* antes de usar datos reales.

7. Componentes principales

7.1 Aplicación frontend

`App.tsx` define las rutas y protege las pantallas internas. Si no existe una sesión, redirige al login.

`Layout.tsx` contiene el menú principal, el indicador de conexión, el usuario activo y el aviso del modo offline.

`lib/api.ts` centraliza las llamadas HTTP: agrega el token JWT, distingue errores de red (`NetworkError`) de errores de la API (`ApiError`), y ante un `401` cierra la sesión. `lib/sync.ts` verifica la conexión cada 8 segundos (`/api/health`), mantiene la cola de ventas pendientes y las sincroniza al volver Internet. `lib/localdb.ts` define las tablas de IndexedDB (catálogo, configuración, `ventasPendientes`).

7.2 API y autenticación

`index.ts` configura Express, CORS, JSON (límite 5 MB), cabeceras de seguridad, rutas y archivos estáticos.

`routes.ts` expone la API bajo el prefijo `/api`:

Grupo	Endpoints principales
Sistema	`GET /health`, `GET /sistema/backup`
Sesión	`POST /auth/login`, `GET /auth/me`, `PUT /auth/perfil`
Productos	`/productos`, `/productos/:id`, `/productos/:id/vencimientos`
Vencimientos	`GET /vencimientos`, `PUT /vencimientos/:id`, `POST /vencimientos/:id/retirar`, `GET /stock-critico`
Ventas	`/ventas`, `POST /ventas/:id/anular`, `POST /sync/ventas`
Caja	`GET /caja/resumen`, `POST /caja/cierre`
Compras	`/intermediario`, `/pedidos` (consolidar, avanzar, duplicar, `pdf`, `excel`), `/ingresos`
Otros	`/dashboard`, `/reportes`, `/importacion/validar`, `/importacion/confirmar`, `/configuracion`, `/alertas/leer`

`auth.ts`: `autenticar()` compara la contraseña con bcrypt; `firmarToken()` genera un JWT de 12 horas (30 días con «Recordarme»); `requireAuth` protege las rutas. No hay roles: cualquier sesión válida accede a todo.

7.3 Acceso a datos

`db.ts` crea el pool de PostgreSQL y contiene helpers para:

- Consultas múltiples (`q`).
- Consulta de un único registro (`q1`).
- Transacciones (`tx`), con `COMMIT` / `ROLLBACK` automáticos.
- Lectura de configuración (`getConfig`).
- Conversión de tipos numéricos (NUMERIC → number) y fechas (DATE → string).

Las operaciones que modifican varias tablas se ejecutan dentro de transacciones para mantener la consistencia.

7.4 Servicios de dominio

Servicio	Responsabilidades
`dominio.ts`	Precio sugerido según costo y margen; estado de stock crítico; aumento y reducción de stock; estados de vencimiento (R6); descuento por vencimiento más próximo (R4); retiro de mercadería vencida (R7)
`ventas.ts`	`registrarVenta`, `anularVenta`; valida, unifica renglones repetidos, calcula total y vuelto, descuenta stock y sincroniza ventas offline de forma idempotente
`caja.ts`	Totales diarios, separación efectivo / digital, diferencia y cierre (R11)
`pedidos.ts`	Consolidación de faltantes (R8), ciclo de vida de los 7 estados (R9), duplicado, historial
`ingresos.ts`	Ingreso de mercadería (R10): suma stock, actualiza costo, precio sugerido y vencimiento; finaliza el pedido
`importacion.ts`	Validación fila por fila y confirmación de la importación
`exportar.ts`	`generarPdfPedido` (PDFKit, A4) y `generarExcelPedido` (ExcelJS)

Reglas de negocio clave:

Regla	Descripción
R1	Stock crítico: `stock_actual <= stock_minimo`
R2	Precio sugerido = `costo × (1 + margen/100)`, redondeado al peso
R3	La venta es transaccional
R4	Se descuenta el vencimiento vigente más próximo, pasando al siguiente si no alcanza
R5	Alerta inmediata al dejar un producto en stock crítico
R6	Estados de vencimiento NORMAL / PROXIMO / VENCIDO / RETIRADO
R7	El retiro descuenta el stock y marca el vencimiento como retirado
R8	Consolidación de faltantes en un pedido; cantidad sugerida `max(mín × 2 − stock, 1)`
R9	Ciclo de vida del pedido, estricto y lineal
R10	Ingreso de mercadería sobre un pedido ENTREGADO o manual
R11	Efectivo esperado = efectivo + fondo; diferencia = contado − esperado; un cierre por día
R12	La anulación devuelve el stock y no se permite con la caja cerrada

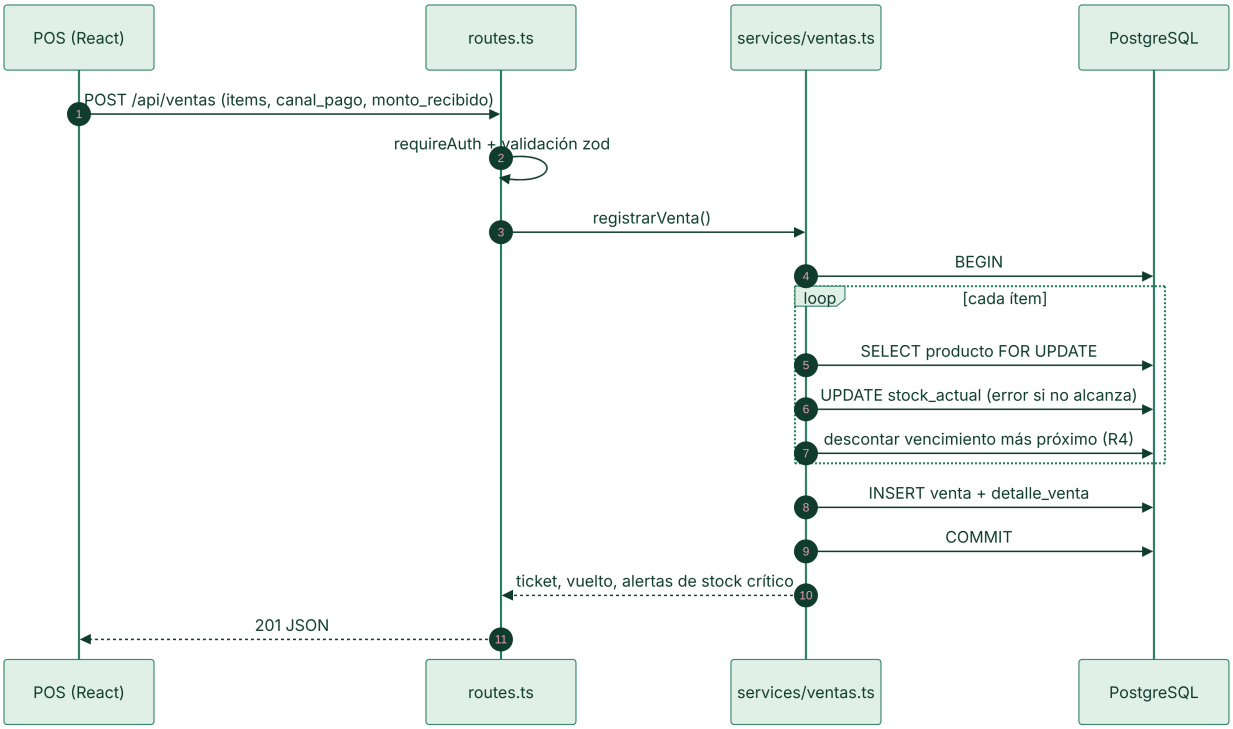
Pedido de compra — 7 estados: `EN_PREPARACION → GENERADO → EXPORTADO → ENVIADO → EN_GESTION → ENTREGADO → FINALIZADO`. Se pasa a `EXPORTADO` al descargar el PDF o el Excel, y a `FINALIZADO` al registrar el ingreso.

7.5 Modo offline

El service worker (vite-plugin-pwa) guarda la aplicación; IndexedDB guarda el catálogo y la configuración (se cargan al ingresar con conexión). Las ventas hechas sin conexión se guardan en `ventasPendientes` con `uuid_cliente` y un ticket `OFF-000X`. Al detectar conexión, `POST /api/sync/ventas` las envía; el servidor las inserta de forma **idempotente** respetando precio y fecha originales. Si el stock del servidor no alcanza, la venta se registra igual (ya fue cobrada), el stock queda en 0 y se crea una `alerta_sistema`. Solo se sincronizan ventas; el

resto de las operaciones requiere conexión. Para pruebas existe un interruptor que simula la caída (`localStorage.gfl_force_offline`, desde Configuración).

8. Flujo técnico de una venta



Secuencia de `POST /api/ventas`.

1. El frontend arma el carrito y envía los productos y el medio de pago.
2. El backend valida el cuerpo con `ventaSchema`.
3. Se bloquean los productos involucrados dentro de una transacción.
4. Se unifican líneas repetidas del mismo producto.
5. Se valida que la cantidad sea compatible con `UNIDAD` o `GRANEL`.
6. Se valida el stock disponible.
7. Se calcula el subtotal, total y vuelto.
8. Se inserta la cabecera en `venta`.
9. Se insertan los registros en `detalle_venta`.
10. Se reduce `producto.stock_actual`.
11. Se descuenta el vencimiento vigente más próximo.
12. Se generan alertas si el stock queda crítico.
13. Se confirma la transacción y se devuelve el resultado al frontend.

La sincronización offline utiliza `uuid_cliente` para evitar duplicar una venta reenviada.

9. Gestión de errores y validaciones

9.1 Formato de errores de la API

Todos los errores se devuelven como JSON `{ "error": "mensaje", "detalle": … }` mediante `errorMiddleware` (`http.ts`). Los handlers asíncronos se envuelven con `ah()` para que ninguna excepción quede sin capturar.

Situación	Código HTTP	Origen
Datos inválidos (esquema Zod)	400	`ZodError` → `{ campo, mensaje }` por cada problema
Regla de negocio incumplida (p. ej. dinero recibido menor al total)	400	`bad()`
JSON mal formado	400	`entity.parse.failed`
Sin sesión o sesión vencida	401	`requireAuth`
Recurso inexistente	404	`notFound()`; las rutas `/api` desconocidas devuelven «Ruta inexistente»
Conflicto de estado (caja cerrada, pedido ya finalizado, stock insuficiente…)	409	`conflict()`
Valor duplicado (`23505`) o registro referenciado (`23503`) en PostgreSQL	409	Traducción de errores de la base
Error no previsto	500	«Error interno del servidor»; el detalle se registra en el log, no se expone

9.2 Validación de datos

- **Backend:** esquemas Zod en cada endpoint (por ejemplo `ventaSchema`: `canal_pago` enumerado, `cantidad` positiva, `id_producto` entero, `uuid_cliente` UUID, `monto_recibido` ≥ 0). Las reglas de dominio se revalidan dentro de la transacción: cantidades enteras para `UNIDAD`, stock suficiente, pedidos sin productos repetidos, fechas `AAAA-MM-DD`, contraseña actual al cambiar la cuenta.
- **Base de datos:** `CHECK`, `NOT NULL`, `UNIQUE` y claves foráneas como última barrera de integridad.
- **Importación de catálogo:** `validarCatalogo` revisa cada fila (código, nombre y categoría no vacíos; precio numérico mayor a 0; modalidad Unidad o Granel; costo, stock y stock mínimo no negativos; códigos duplicados dentro del archivo). Las filas con error no se importan y se informan con su número de fila.
- **Frontend:** los formularios validan antes de enviar y muestran avisos (por ejemplo, contraseñas distintas, valores numéricos inválidos).

9.3 Errores de conexión y situaciones excepcionales

- **Sin conexión con el servidor:** el cliente HTTP lanza `NetworkError` (también ante 502, 503 y 504). La interfaz pasa a modo **Sin conexión**, el POS sigue vendiendo con la cola local y las demás pantallas quedan en solo lectura con datos en caché.
- **Pantalla sin datos tras un fallo:** se muestra un cuadro de error con **Reintentar**; en Configuración, la carga se reintenta sola al volver la conexión.
- **Sesión vencida (401):** el cliente cierra la sesión y vuelve al login.
- **Base de datos caída:** las consultas fallan con 500 o error de red; el pool reintenta en la siguiente solicitud. Revisar los logs del servicio `app` y de `db`.

- **Concurrencia:** las ventas bloquean las filas de producto (`SELECT … FOR UPDATE`) dentro de una transacción; ante cualquier error se hace `ROLLBACK` y no queda stock descontado a medias.
- **Reenvío de ventas offline:** `uuid_cliente` único evita duplicados.
- **Stock insuficiente al sincronizar:** se acepta la venta, el stock queda en 0 y se registra una alerta del sistema.
- **Operaciones bloqueadas por regla:** anulación o segundo cierre con caja cerrada, edición de pedidos ya generados, ingreso de pedidos no entregados (HTTP 409 con mensaje claro).

10. Pruebas implementadas

10.1 Prueba de integración de la API (`e2e.ts`)

Se encuentra en `backend/src/test/e2e.ts`. Utiliza PostgreSQL real, carga datos de demostración, levanta la aplicación en un puerto dinámico y prueba la API completa.

```
cd backend
DATABASE_URL=postgres://... npm test
```

Atención: la prueba recarga los datos de demostración; no ejecutarla sobre una base con datos reales.

Resultado obtenido (17 grupos, todos correctos):

```
✓ health y login
✓ catálogo de demostración
✓ R3/R4 venta: descuenta stock y el vencimiento más próximo; vuelto
✓ R3 un renglón repetido se unifica y baja el vencimiento siguiente al agotar el primero
✓ validaciones de venta
✓ sync offline: idempotente, respeta precio y fecha; stock insuficiente deja 0 y avisa
✓ R12 anular venta devuelve el stock
✓ vencimientos: filtros, estados y R7 retirar
✓ R8 consolidar faltantes en UN pedido al intermediario único
✓ R9 ciclo de vida estricto; PDF y Excel; edición bloqueada
✓ R10 ingreso de mercadería desde pedido ENTREGADO
✓ duplicar pedido
✓ R11 caja: resumen, arqueo, cierre único y anulación bloqueada
✓ reportes y dashboard
✓ CU-09 importación de catálogo con errores
✓ productos: alta, precio sugerido, baja lógica y configuración
✓ intermediario único, perfil y backup

17 grupos de pruebas OK
```

10.2 Prueba de interfaz de punta a punta

Se ejecutó un recorrido de 26 pasos en Chromium real con Playwright sobre el sistema compilado: login correcto e incorrecto, venta con producto a granel y lector de código, stock insuficiente, alerta de stock crítico, modo offline y sincronización, retiro de vencimientos, pedido con sus 7 estados, descargas reales de PDF y Excel, ingreso de mercadería, caja, reportes, importación y ausencia de errores de JavaScript en consola. Este recorrido sirvió también para generar las capturas del Manual de usuario. El script no forma parte del repositorio.

10.3 Pruebas unitarias

El repositorio actual no contiene una suite separada de pruebas unitarias con Jest, Vitest u otro runner. Las funciones de dominio, como el cálculo de precio y la clasificación de stock, están separadas y son candidatas directas para pruebas unitarias futuras.

Por lo tanto, para una presentación académica debe distinguirse:

- `e2e.ts`: prueba de integración/end-to-end contra API y PostgreSQL.
- Recorrido de interfaz con Playwright: prueba manual automatizada, fuera del repositorio.
- Pruebas unitarias aisladas: no incluidas actualmente.

También se verificó el arranque compilado de producción (`node dist/…`). La imagen Docker debe probarse con `docker compose up --build` en el entorno de despliegue.

11. Datos de demostración

`seed.ts` crea:

- Usuario `admin` con contraseña `1234`.
- Productos de ejemplo.
- Vencimientos.
- Ventas históricas.
- Pedidos e intermediario.
- Configuración inicial.

`npm run seed` borra y reemplaza los datos de negocio. El arranque Docker utiliza `seed --if-empty`, por lo que solamente carga la demostración cuando la base todavía no tiene usuarios.

12. Consideraciones de mantenimiento

- No ejecutar `npm run seed` sobre una base productiva.
- Mantener protegidos `JWT_SECRET`, contraseñas y archivos `.env`.
- Hacer respaldos de PostgreSQL con `pg_dump` y del volumen de almacenamiento.
- Ejecutar migraciones antes de iniciar una nueva versión; las nuevas se agregan como archivos numerados en `backend/migrations/`.
- Validar los builds del frontend y backend (`npm run build`, `npm run typecheck`) antes de desplegar.
- Si se expone a Internet, colocar el sistema detrás de HTTPS (proxy inverso).
- Revisar los logs del servicio `app` ante fallos de inicio.
- Evolución sugerida: pruebas unitarias de `dominio.ts`, respaldo programado con `pg_dump` y réplica remota de la base.